SentinelGuard — Continuity Document

Hand this to a new Claude chat to continue exactly where we left off.

Who I am

- **Name:** Shubham (GitHub: Ayush-Anand701)
- **Role:** Student and software developer — first real project
- **Goal:** Build SentinelGuard as a resume project, open-source library, research paper, and patent filing

---

What SentinelGuard is

An open-source Python middleware library that detects **multi-turn prompt injection attacks** in LLM applications.

The core idea in one sentence: Every existing tool (LlamaFirewall, PromptGuard, etc.) scores each message in isolation. SentinelGuard maintains memory of the entire conversation and detects when intent *drifts gradually* toward an attack across multiple turns — catching attacks that no single-message detector can see.

The patent claim:

> "A method for detecting adversarial intent drift in multi-turn LLM conversations using sliding-window semantic embedding comparison across conversation history, combined with Peak+Accumulation risk scoring and per-deployment adaptive threshold calibration."

Why it's novel:

- Multi-turn attacks succeed 92.78% of the time against ALL existing tools (joint OpenAI/Anthropic/DeepMind study, 2025)
- LlamaFirewall confirmed bypassed by Unicode invisible characters (Trendyol, May 2025) — unpatched
- DeepContext (arXiv 2602.16935) is the closest research paper but: not open source, no pip package, no public benchmark
- No pip-installable multi-turn defense exists anywhere today

---

Tech stack

Layer	Tool
Language	Python 3.11+
Embeddings	sentence-transformers (all-MiniLM-L6-v2)
API / Middleware	FastAPI
Testing	pytest
Packaging	pyproject.toml + hatchling
Version control	Git + GitHub

Project folder structure

```
sentinelguard/
├── src/sentinelguard/
│   ├── __init__.py
│   ├── normalizer.py        ← Phase 02 — BUILD THIS NEXT
│   ├── drift_detector.py    ← Phase 03
│   ├── risk_scorer.py       ← Phase 04
│   ├── feedback.py          ← Phase 04
│   └── middleware.py        ← Phase 05
├── tests/
│   ├── __init__.py
│   ├── test_normalizer.py
│   ├── test_drift.py
│   └── test_integration.py
├── benchmarks/datasets/
├── notebooks/
├── docs/
│   └── research_notes.md
├── pyproject.toml
└── README.md
```

GitHub repo: https://github.com/Ayush-Anand701/sentinelguard **Branch:** main **Virtual environment:** .venv (Windows: activate with `.venv\Scripts\activate`)

Exact progress — what is DONE ✅

Phase 01 — Environment setup & research groundwork ✅ COMPLETE

Environment (all done):

- ☑ sentinelguard folder created on Windows machine
- ☑ Python virtual environment (.venv) created and activated
- ☑ All dependencies installed: sentence-transformers, numpy, fastapi, uvicorn, pytest, black
- ☑ Full folder structure created (src/, tests/, benchmarks/, docs/, notebooks/)
- ☑ All placeholder .py files created
- ☑ Git initialised, .gitignore set up
- ☑ Pushed to GitHub (branch: main, using Personal Access Token for auth)
- ☑ GitHub Actions CI configured

Research (all done):

- ☑ Paper 1 read: DeepContext (arXiv 2602.16935) — F1=0.84, RNN hidden state architecture, "glass cannon effect" limitation noted
- ☑ Paper 2 read: Peak+Accumulation (arXiv 2602.11247) — full formula documented with all constants (ρ=0.45, δ=0.15, βe=0.2, βr=0.7, threshold=0.7)
- ☑ Paper 3 read: Crescendo (Russinovich et al. 2024) — ~5 turns typical, drift visible from turn 3-4
- ☑ research_notes.md filled and saved in docs/ folder
- ☑ Manual attack experiments run on GPT-4o (Crescendo, DAN/persona, fictional framing, gradual erosion)

Other completed tasks:

- ☑ AI Lab Project Review Document generated (.docx, all 5 sections filled, 11 related works in table)
- ☑ Resume bullet points written for SentinelGuard
- ☑ Professor meeting prep done (patent angle, Q&A, meeting tips)

What is NEXT ▶️

Phase 02 — Build the UnicodeNormalizer (Week 2)

This is the very next thing to build. It's the first actual Python code in the project.

File to edit: `src/sentinelguard/normalizer.py`

What it does: Strips Unicode invisible characters, normalises homoglyphs, and cleans text before any classifier sees it. Closes the confirmed LlamaFirewall Unicode bypass (May 2025).

Specific Unicode characters to strip:

- U+200B (zero-width space)
- U+FEFF (byte order mark)
- U+00AD (soft hyphen)
- U+200C, U+200D (zero-width non-joiner/joiner)
- Apply NFKC normalisation for homoglyphs (Cyrillic 'а' → Latin 'a' etc.)

The class should look like this (skeleton):

python

```python
import unicodedata
import re

class UnicodeNormalizer:
    def normalize(self, text: str) -> str:
        # 1. Strip zero-width characters
        # 2. NFKC normalize
        # 3. Map homoglyphs
        # 4. Return clean text
        pass

    def get_audit_log(self) -> list:
        # Return list of what was stripped and where
        pass
```

Test file to fill: `tests/test_normalizer.py`

- Needs 50+ test cases covering: invisible char injections, homoglyphs, base64 tricks, benign messages that must pass unchanged
- Latency benchmark: must be under 2ms per message

The guide Shubham is following: Step-by-step, small instructions, copy-paste commands, explicit "what you should see" after every step. He is a beginner — explain every command in plain English before asking him to run it.

Full 6-phase roadmap (for reference)

Phase	What	Timeline	Status
01	Environment + research	Week 1	✅ Done
02	UnicodeNormalizer	Week 2	▶️ Next
03	IntentDriftDetector	Weeks 3-4	⏳ Pending
04	RiskScorer + FeedbackLoop	Week 5	⏳ Pending
05	pip library + FastAPI middleware	Week 6	⏳ Pending
06	Benchmark + paper + patent	Weeks 7-8	⏳ Pending

Key decisions already made

Decision	What was decided
Embedding model	all-MiniLM-L6-v2 (dev), all-mpnet-base-v2 (production)
Sliding window size	Default 10 turns
Risk thresholds	LOW <0.35, MEDIUM 0.35-0.65, HIGH >0.65
Scoring formula	Peak+Accumulation: score = 0.6×peak + 0.4×sum
Branch name	main (not master)
GitHub auth	Personal Access Token (PAT) stored in remote URL
Patent type	Provisional patent (USPTO, ~$320, file in Week 8)

Important technical context

Why the branch is "main" not "master": When git status showed "On branch master" initially, the actual push succeeded on "main". Always use `git push origin main`.

How to reactivate the virtual environment (do this every time VS Code is reopened):

```
.venv\Scripts\activate
```

You should see `(.venv)` at the start of the terminal line.

How to push to GitHub after any work session:

```
git add .
git commit -m "describe what you did"
git push
```

Shubham's preferences for how Claude should guide him

1. **Small steps** — break every task into the tiniest possible sub-steps
2. **Copy-paste commands** — every terminal command should have a copy button or be in a code block
3. **"What you should see"** — after every command, tell him exactly what output to expect
4. **Plain English explanations** — before every command, explain WHY in one sentence
5. **Beginner-friendly** — he is new to professional software development, don't assume knowledge
6. **Confirm before continuing** — ask him to paste his terminal output before moving to the next step

---

How to continue in a new chat

Paste this entire document into a new Claude chat, then say:

> "I'm Shubham. This is my continuity document for the SentinelGuard project. I've completed Phase 01 including the manual attack experiments. Please guide me through Phase 02 — building the UnicodeNormalizer — in the same small step-by-step style as before."

Claude will pick up exactly where we left off.

Generated at end of Phase 01 — September 2026